LAYER 4 · TRANSPORT

L4

Ports, Handshakes and Firewalls

How data reaches the right program on the right machine: TCP and UDP, ports and sockets, the handshake, firewall rules and reading connections with netstat.

YASIR ABDULLAH · YASIR-PORTFOLIO-ZETA.VERCEL.APP/NOTES/EBOOKS/LAYER-4-TRANSPORT

SKIP TO CONTENT

Contents

1 **TCP vs UDP**

Reliable and ordered, or fast and fire-and-forget: choosing between the two transports.

2 **Ports and Sockets**

How one machine keeps many conversations apart: address plus port, at both ends.

3 **The Three-Way Handshake**

SYN, SYN-ACK, ACK: how TCP opens a connection, and how it closes one.

4 **Firewall Rules**

Allowing only the traffic you mean to, and why replies get through a stateful firewall.

5 **netstat**

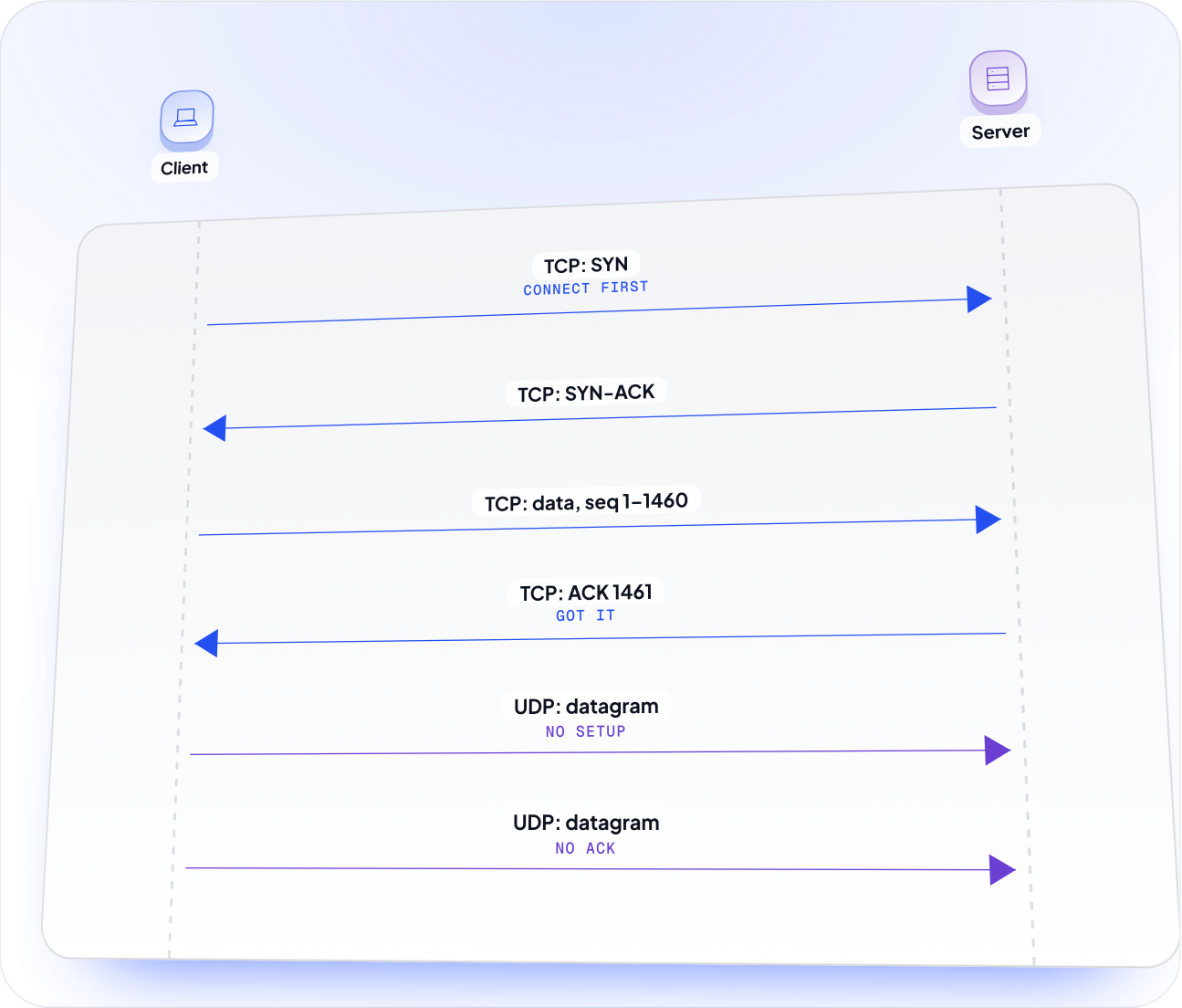
Listing open ports and live connections, and which program owns each one.

SKIP TO CONTENT

CHAPTER 1 · TCP VS UDP

TCP vs UDP

Reliable and ordered, or fast and fire-and-forget: choosing between the two transports.



TCP sets up a connection and acknowledges what arrives. UDP just sends; any checking is the application's job.

Once a packet reaches the right machine, the transport layer gets it to the right program. There are two main ways to do that, and they make opposite trade-offs.

TCP: reliable and ordered

SKIP TO CONTENT

ntrol Protocol) sets up a connection before sending anything, numbers every byte and expects the other side to acknowledge what it receives.

- **Connection:** a three-way handshake opens it, FINs close it.
- **Reliability:** anything not acknowledged in time is sent again.
- **Order:** data reaches the application in the order it was sent.
- **Flow and congestion control:** the sender slows down when the receiver or the network cannot keep up.

The cost is overhead and delay: a lost packet holds up everything behind it until it is resent.

UDP: fast and simple

UDP (User Datagram Protocol) just sends datagrams. No handshake, no acknowledgements, no ordering, no resending. Its header is only 8 bytes against TCP's 20 or more.

If the application needs reliability, it builds its own. Many do, in lighter ways that suit them.

Side by side

	TCP	UDP
Connection	Yes, handshake first	No
Delivery	Guaranteed or error	Best effort
Order	Kept	Not kept
Speed	Slower to start, steady	Immediate
Typical uses	Web (HTTP/1.1, HTTP/2), email, SSH, file transfer	DNS, DHCP, VoIP, vide

Choosing

- If every byte must arrive and in order, such as a file, a web page or a database query, use **TCP**.
- If late is useless, such as a voice packet from half a second ago, use **UDP**: it is better to skip it than to wait.

SKIP TO CONTENT

HTTP/3 is a good example of the modern middle ground. It runs QUIC over UDP and adds its own reliability per stream, so one lost packet no longer stalls the whole page.

Seeing it

WINDOWS CMD

```
netstat -ano -p tcp
netstat -ano -p udp
```

TCP sockets show states such as `ESTABLISHED` and `LISTENING`. UDP sockets have no state at all, because there is no connection.

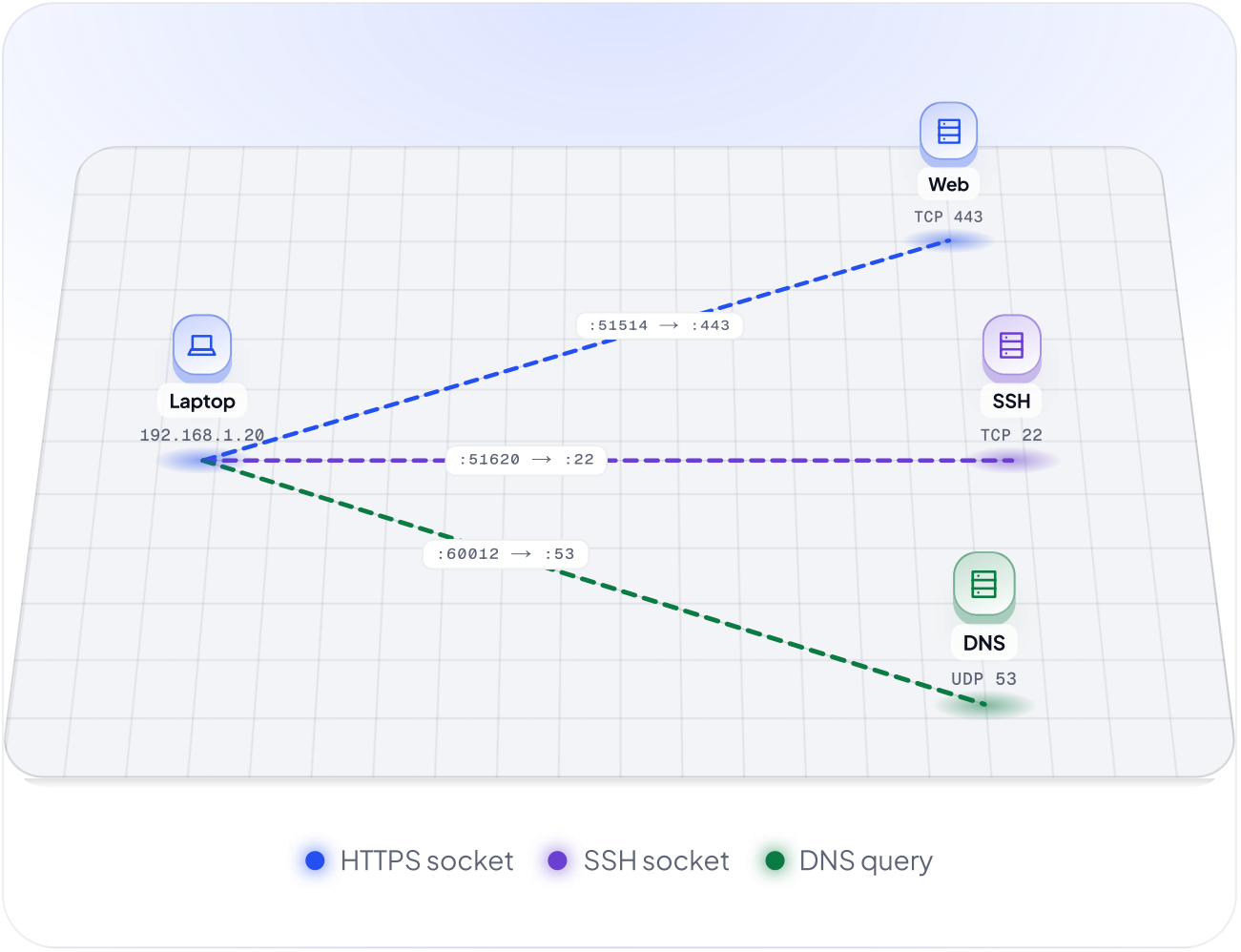
When it goes wrong

- **A UDP service "works sometimes":** a firewall allows TCP on that port but not UDP. DNS over UDP 53 is the classic case.
- **Video call choppy, downloads fine:** packet loss or jitter. TCP hides it by resending; real-time UDP cannot.

SKIP TO CONTENT

Ports and Sockets

How one machine keeps many conversations apart: address plus port, at both ends.



One laptop, three conversations. Each is a socket pair: its own random port on the left, a well-known port on the right.

An IP address gets data to a machine. A **port** gets it to the right program on that machine. Port numbers run from 0 to 65535, separately for TCP and for UDP.

Well-known ports

Servers listen on agreed ports so clients know where to knock.

SKIP TO CONTENT

PORT	PROTOCOL	SERVICE
20, 21	TCP	FTP
22	TCP	SSH
25	TCP	SMTP (mail between servers)
53	UDP/TCP	DNS
67, 68	UDP	DHCP
80	TCP	HTTP
123	UDP	NTP (time)
443	TCP/UDP	HTTPS (UDP for HTTP/3)
3389	TCP	Remote Desktop
5432	TCP	PostgreSQL
6379	TCP	Redis

Ports 0 to 1023 are the well-known range. The client side uses a random **ephemeral** port, usually 49152 to 65535 on Windows.

Sockets

A socket is one end of a conversation: an IP address plus a port. A TCP connection is identified by **both** ends, plus the protocol:

```
TEXT
192.168.1.20:51514  ↔  93.184.216.34:443   (TCP)
```

That pair is why a web server can talk to thousands of clients on the same port 443 at once. Each connection has a different client address or port, so each pair is unique. It is also why one laptop can open ten tabs to the same site: each tab gets its own

SKIP TO CONTENT

Listening

A server program opens a socket and **listens** on its port. If nothing is listening, a TCP connection attempt gets an immediate refusal.

Test whether a port is reachable:

```text
Test-NetConnection example.com -Port 443
```

```shell
nc -vz example.com 443
```

Finding what owns a port

```windows cmd
netstat -ano | findstr :443
tasklist /fi "PID eq 4120"
```

```shell
ss -tlnp
```

When it goes wrong

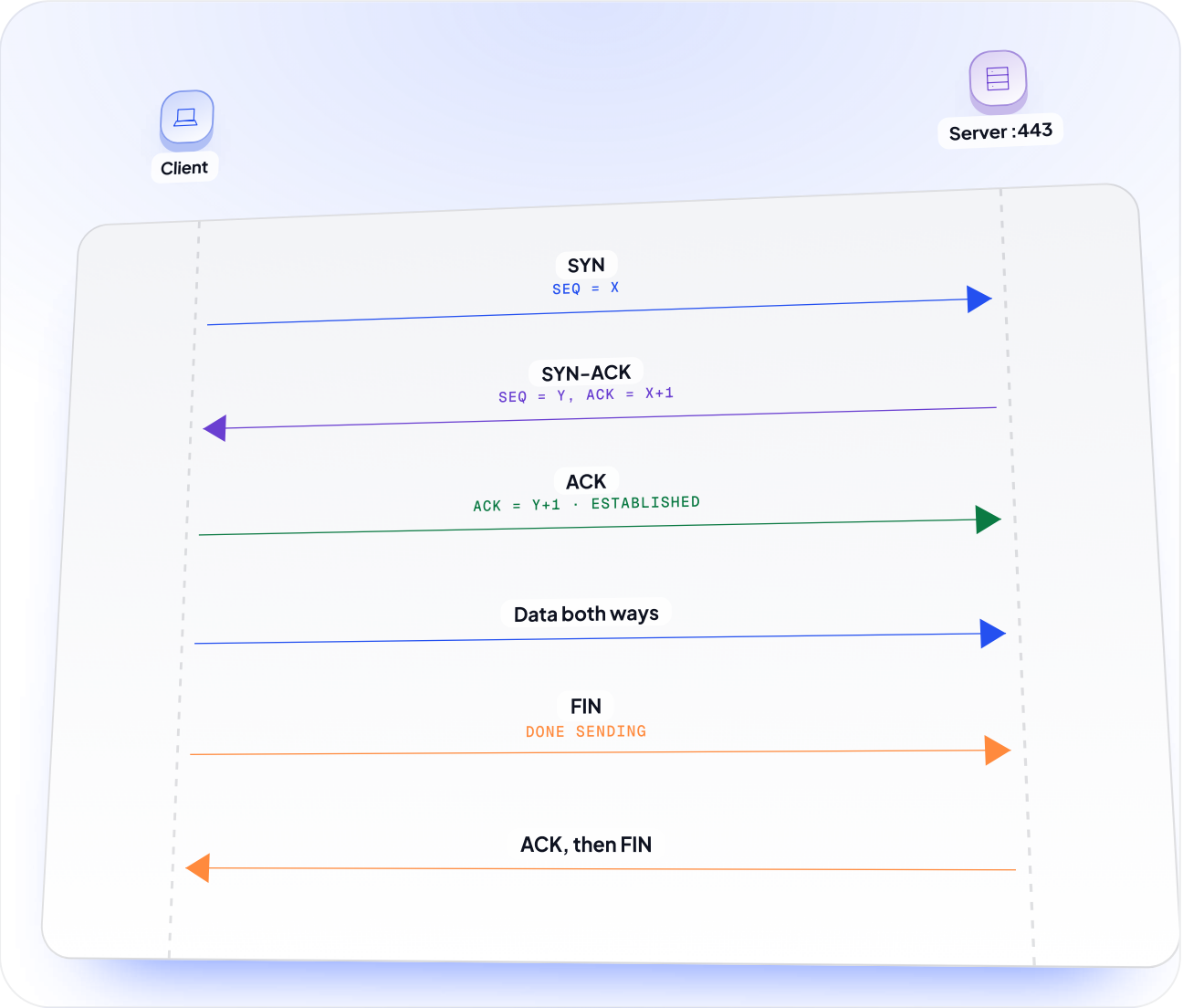
- **"Connection refused":** the machine answered, but nothing is listening on that port.
- **"Connection timed out":** nothing answered at all; usually a firewall dropping the packets, or the wrong address.
- **"Address already in use" when starting a service:** another program already holds that port. Find it with netstat and stop it, or pick another port.

SKIP TO CONTENT

CHAPTER 3 · THREE-WAY HANDSHAKE

The Three-Way Handshake

SYN, SYN-ACK, ACK: how TCP opens a connection, and how it closes one.



Both sides pick a starting sequence number and confirm the other's. Closing takes a FIN and an ACK in each direction.

Before TCP sends a single byte of data, both sides agree to talk and agree on where their byte numbering starts. That agreement takes three packets.

Opening a connection

ks a random initial sequence number, x, and sends a segment with

SKIP TO CONTENT

2. **SYN-ACK:** the server picks its own number, y, and replies with SYN and ACK set, acknowledging x + 1.
3. **ACK:** the client acknowledges y + 1.

Both sides are now **ESTABLISHED**. Each knows the other is reachable, and each knows the other's starting sequence number, so every later byte can be counted and acknowledged.

Random starting numbers make it hard for an attacker to inject fake segments into someone else's connection.

Closing a connection

Each direction closes on its own:

1. The side that has finished sends **FIN**.
2. The other side **ACK**s it, sends any last data, then sends its own **FIN**.
3. The first side **ACK**s that.

The side that closed first then waits in `TIME_WAIT` for a short while, so any stray late segments die out before the same port pair is reused. Seeing many `TIME_WAIT` lines on a busy server is normal.

A connection can also be cut abruptly with **RST** (reset), for example when a packet arrives for a port where nothing is listening.

Watching it happen

```shell
sudo tcpdump -n -i any 'tcp port 443 and (tcp[tcpflags] & (tcp-syn|tcp-fin|
```

In Wireshark, the filter `tcp.flags.syn == 1` shows only connection attempts. A SYN with no SYN-ACK in reply means the server never answered.

What the handshake tells you when troubleshooting

SKIP TO CONTENT

WHAT YOU SEE	LIKELY CAUSE
SYN, no reply, SYN again	Firewall dropping it, or the server is unreachable
SYN, then RST	Server reachable, nothing listening on that port
Handshake completes, then stall	Application problem, or MTU issue on large packets
Many half-open SYN_RECEIVED	A SYN flood, or clients that cannot get replies back

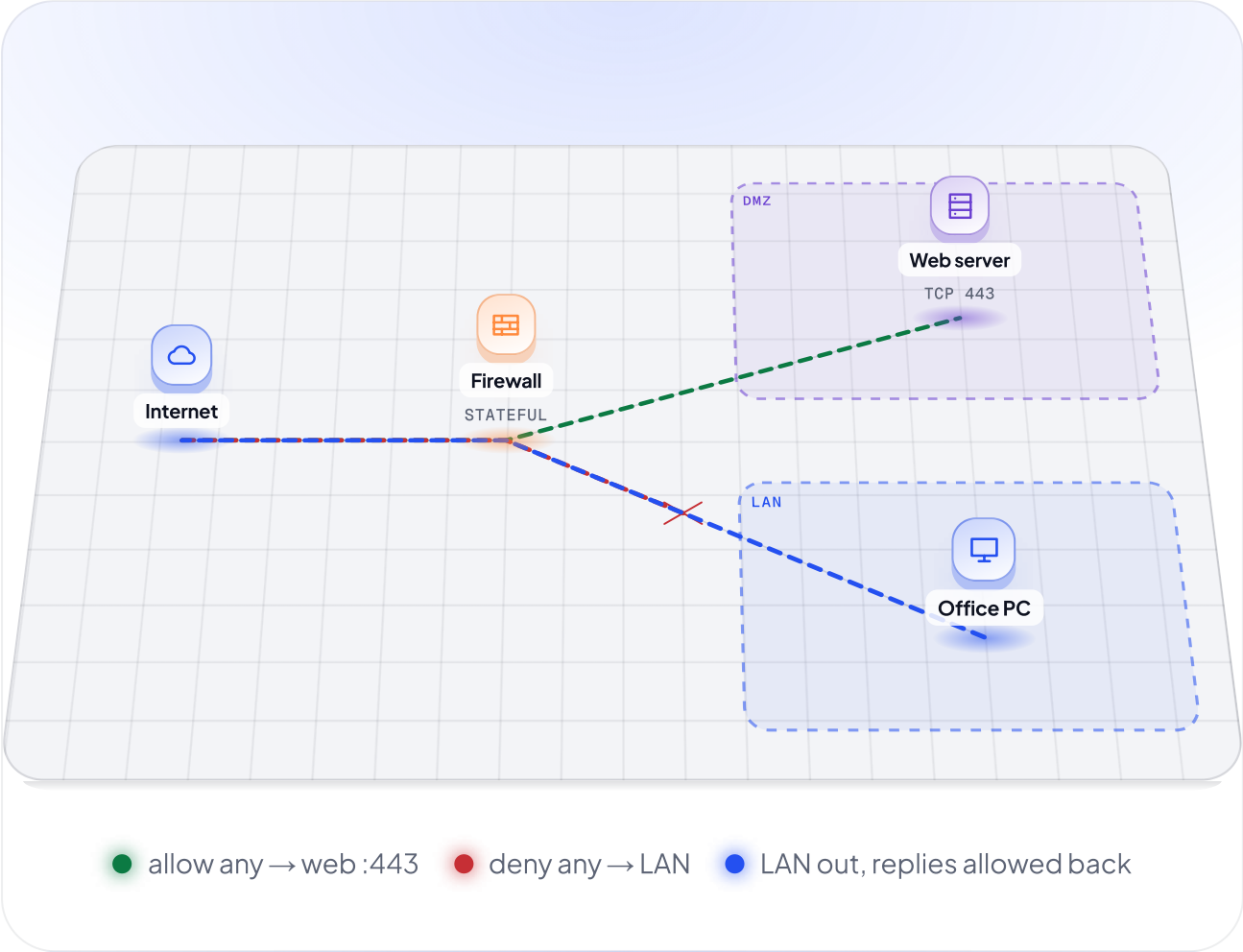
TIP

"Connection refused" and "connection timed out" are not the same problem. Refused means the handshake got an RST: the host is there. Timed out means nothing came back at all.

SKIP TO CONTENT

Firewall Rules

Allowing only the traffic you mean to, and why replies get through a stateful firewall.



Outside may reach the web server on 443 and nothing else. Inside may go out, and the replies are let back in because the firewall remembers the request.

A firewall decides which connections may pass between networks. Its rules usually look at source, destination, protocol and port, and are read top to bottom like an ACL. What makes a modern firewall different is that it is **stateful**.

Stateful means it remembers

When a PC opens a connection to a website, the firewall records it in a **state** table. Any reply from that website matches that entry and is allowed back in automatically. You never write a rule for return traffic.

SKIP TO CONTENT

An unsolicited packet from the internet matches no entry, so it is checked against the inbound rules, and usually dropped.

Default deny

Start from "block everything", then allow only what has a reason:

#	FROM	TO	SERVICE	ACTION
1	Internet	Web server	TCP 443	Allow
2	LAN	Internet	TCP 80, 443, UDP 53	Allow
3	Admin PC	Web server	TCP 22	Allow
4	Any	Any	Any	Deny, log

Put public servers in a **DMZ**, a separate zone, so that if one is compromised it still cannot reach the LAN.

Host firewalls

Every server and PC should also filter its own ports.

TEXT

```
New-NetFirewallRule -DisplayName "Allow HTTPS" -Direction Inbound -Protocol
Get-NetFirewallRule -Enabled True -Direction Inbound | Select DisplayName
```

SHELL

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered
```

SKIP TO CONTENT

On a remote Linux server, allow SSH **before** enabling the firewall, or you lock yourself out.

Good habits

- Name every rule and add a comment saying who asked for it and why.
- Allow specific sources wherever you can; "any" is the last resort.
- Log denied traffic, and read the log when something new breaks.
- Review rules every few months and remove the ones nobody can explain.

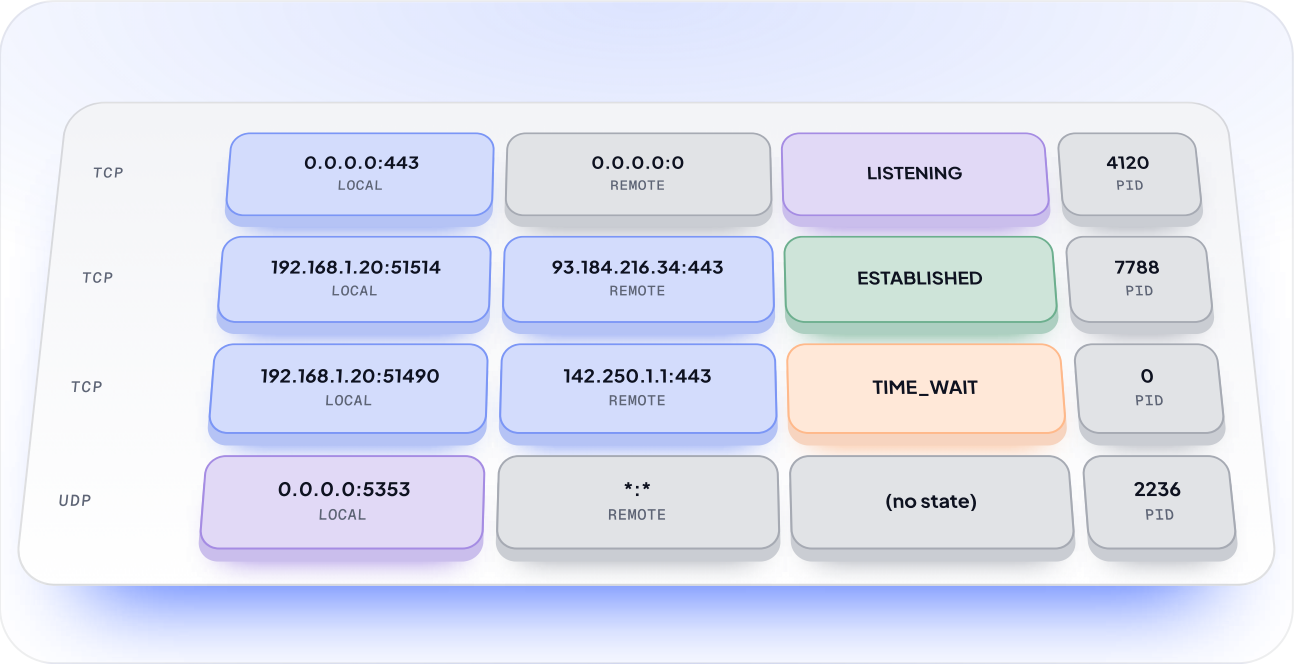
When it goes wrong

- **Works inside, not from outside:** the inbound rule or the port forward is missing.
- **Works by IP, not by name:** outbound DNS (UDP and TCP 53) is blocked.
- **One app fails after a firewall change:** check the deny log for its traffic; it often needs an extra port nobody wrote down.

SKIP TO CONTENT

netstat

Listing open ports and live connections, and which program owns each one.



netstat -ano on Windows: each line is a socket, its state and the process ID that owns it.

`netstat` lists every socket on a machine: what is listening, what is connected to where, the state of each connection and, with the right flag, which program owns it. It is the quickest way to answer "is this service even running?"

The flags you need on Windows

WINDOWS CMD

```
netstat -ano
```

- `-a` all sockets, including listening ones
- `-n` numbers, not names (much faster)
- `-o` the owning process ID (PID)

Add `-b` (in an administrator prompt) to see the program name directly.

SKIP TO CONTENT

Reading the output

```text
  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:443            0.0.0.0:0              LISTENING       4120
  TCP    192.168.1.20:51514     93.184.216.34:443      ESTABLISHED     7788
  TCP    192.168.1.20:51490     142.250.1.1:443        TIME_WAIT       0
  UDP    0.0.0.0:5353           *:*                                    2236
```

- **0.0.0.0** as a local address means "listening on every interface". **127.0.0.1** means local only, unreachable from other machines.
- `LISTENING`: a service waiting for connections.
- `ESTABLISHED`: a live connection.
- `TIME_WAIT`: recently closed; normal, and it clears on its own.
- `SYN_SENT` that never changes: this machine is trying to connect and getting no reply.
- `CLOSE_WAIT` piling up: the other side closed, but the local program never did. That is usually a bug in the program.

Useful combinations

Is anything listening on port 3389?

```WINDOWS CMD
netstat -ano | findstr :3389
```

Which program is PID 4120?

```WINDOWS CMD
tasklist /fi "PID eq 4120"
```

Count connections by state:

```text
Get-NetTCPConnection | Group-Object State | Select Name, Count
```

SKIP TO CONTENT

On Linux

`ss` has replaced netstat:

```shell
ss -tlnp      # TCP listening sockets, with the process
ss -tunap     # all TCP and UDP sockets
```

Typical support uses

- **"The app can't connect to the server":** on the server, check the service is `LISTENING` on the expected port and not just on 127.0.0.1.
- **"Port already in use":** find the PID holding it and decide whether to stop it.
- **A PC talking to strange addresses:** list `ESTABLISHED` connections with their PIDs and identify each program.